

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..20..8

CUSTOMIZATIONS

 Copy as Markdown

Customize Google sign-in

This page covers configuration and UI options for the Google identity provider beyond the basic setup. On the server, you can control how the client secret is loaded and hook into the sign-in flow with callbacks. In the app, you can configure client IDs and redirect URIs, customize the sign-in button with the `GoogleSignInWidget`, or build a completely custom authentication interface with the `GoogleAuthController`.

Server configuration

These options control how the Google identity provider behaves on the server.

Configuration options

Below is a non-exhaustive list of some of the most common configuration options. For more details on all options, check the `GoogleIdpConfig` in-code documentation.

The Google identity provider can be configured using one of two classes:

- `GoogleIdpConfigFromPasswords`: Automatically loads the client secret from the `googleClientSecret` key in `passwords.yaml` (or the `SERVERPOD_PASSWORD_googleClientSecret` environment variable). This is the class used in the [setup guide](#) and is recommended for most projects.
- `GoogleIdpConfig`: Requires you to pass a `GoogleClientSecret` object directly. Use this when you need to load credentials from a custom source, such as a JSON file, a secrets manager, or a programmatically constructed map.

The `GoogleIdpConfigFromPasswords` class is a convenience wrapper around `GoogleIdpConfig` handles credential loading for you.



Both classes accept the same optional callbacks, such as `googleAccountDetailsValidation` and `getExtraGoogleInfoCallback`, shown below. The examples on this page use `GoogleIdpConfigFromPasswords` unless the section specifically demonstrates manual client secret loading.

Load the client secret using GoogleIdpConfig

When using `GoogleIdpConfig`, you must provide the client secret explicitly.

You can load the secret in several ways:

From JSON string (recommended for production):

```
final googleIdpConfig = GoogleIdpConfig(
  clientSecret: GoogleClientSecret.fromJsonString(
    pod.getPassword('googleClientSecret')!,
  ),
);
```

From JSON file:

```
final googleIdpConfig = GoogleIdpConfig(
  clientSecret: GoogleClientSecret.fromJsonFile(
    File('config/google_client_secret.json'),
  ),
);
```

From JSON map:

```
final googleIdpConfig = GoogleIdpConfig(
  clientSecret: GoogleClientSecret.fromJson({
    'web': {
      'client_id': 'your-client-id.apps.googleusercontent.com',
      'client_secret': 'your-client-secret',
      'redirect_uris': [
        'https://your-domain.com/auth/callback',
      ],
    },
  }),
);
```

Custom account validation

You can customize the validation for Google account details before allowing sign-in. The default validation rejects sign-in unless `verifiedEmail` is true and both `name` and `fullName` are present.

```
final googleIdpConfig = GoogleIdpConfigFromPasswords(
  // Optional: Custom validation for Google account details
  googleAccountDetailsValidation: (accountDetails) {
    // Throw an exception if account doesn't meet custom requirements
    if (accountDetails.verifiedEmail != true ||
        !accountDetails.email.endsWith('@example.com')) {
      throw GoogleUserInfoMissingDataException();
    }
  },
);
```

Accessing Google APIs on the server

On the server side, you can access Google APIs using the access token. The

`getExtraGoogleInfoCallback` in `GoogleIdpConfig` receives the access token and can be used to call Google APIs:

```
import 'package:http/http.dart' as http;

final googleIdpConfig = GoogleIdpConfigFromPasswords(
  // Optional: Extract additional info from Google APIs
  getExtraGoogleInfoCallback: (session, {
    required accountDetails,
    required accessToken,
    required transaction,
  }) async {
    // Use accessToken to call Google APIs and store additional info
    // Example: Access YouTube API
    final response = await http.get(
      Uri.https('www.googleapis.com', '/youtube/v3/channels', {
        'part': 'snippet',
        'mine': 'true',
      }),
      headers: {'Authorization': 'Bearer $accessToken'},
    );
    // Process response and store additional info in the database
  },
);
```

To request additional scopes at sign-in, see [Accessing Google APIs](#) under App configuration.

Reacting to auth user creation

The `onBeforeAuthUserCreated` and `onAfterAuthUserCreated` hooks are global callbacks configured on `AuthUsersConfig` in `initializeAuthServices`. They are not specific to Google. They fire for every identity provider. See [user creation callbacks](#) for full details on both hooks.

GoogleIdpConfig parameter reference

Parameter	Type	Required	Description
<code>clientSecret</code>	<code>GoogleClientSecret</code>	Yes	The Google OAuth client secret loaded from JSON. Can be loaded via <code>fromJsonString</code> , <code>fromJsonFile</code> , or <code>fromJson</code> .
<code>googleAccountDetailsValidation</code>	<code>GoogleAccountDetailsValidation</code>	No	Custom validation callback for Google account details before allowing sign-in. Throws an exception to reject the account.
<code>getExtraGoogleInfoCallback</code>	<code>GetExtraGoogleInfoCallback?</code>	No	Callback that receives the access token after sign-in, allowing you to call additional Google APIs and store extra user data.
<code>onAfterGoogleAccountCreated</code>	<code>AfterGoogleAccountCreatedFunction?</code>	No	Callback invoked after a new Google account has been created and linked to an auth user. Runs inside the same transaction as account creation.
<code>clockSkewTolerance</code>	<code>Duration</code>	No	Tolerance for clock skew when validating Google ID token timestamps. Defaults to the framework's default tolerance.

App configuration

These options are set in your Flutter app rather than on the server.

Accessing Google APIs

The default setup allows access to basic user information, such as email, profile image, and name. You may require additional access scopes, such as accessing a user's calendar, contacts, or files. To do this, you will need to:

- Add the required scopes to the [Data Access](#) page in the Google Auth Platform.
- Request access to the scopes when signing in. Do this by setting the `scopes` parameter of the `GoogleSignInWidget` or `GoogleAuthController`.

For a full list of available scopes, see the [Google OAuth 2.0 Scopes reference](#).

Adding additional scopes may require approval by Google. On the OAuth consent screen, you can see which of your scopes are considered sensitive.

To use the granted scopes from the server with the access token, see [Accessing Google APIs on the server](#).

Lightweight sign-in on the Flutter app

Lightweight sign-in is a feature that attempts to authenticate users previously logged in with Google automatically with minimal or no user interaction. When enabled, the Google authentication controller will try to sign the user in using platform-specific lightweight authentication methods. This feature is disabled by default, but can be enabled from the `GoogleSignInWidget` or `GoogleAuthController`.

```
GoogleSignInWidget(  
  client: client,  
  attemptLightweightSignIn: true, // Enable lightweight sign-in  
  onAuthenticated: () {  
    // User was automatically signed in  
  },  
)
```

NOTE

Lightweight sign-in runs automatically when the controller is initialized (typically at app launch). If it fails (no previous session, or the user dismisses the prompt), the regular sign-in button remains available.

On web, the option has no effect in this version. It only applies to Android and iOS.

Configuring client IDs on the app

If no client IDs are provided programmatically, the underlying `google_sign_in` package falls back to `GoogleService-Info.plist` on iOS. On Android, the `google-services.json` fallback only works when the app uses the Firebase `com.google.gms.google-services` Gradle plugin. A plain project must pass the IDs in code or with `--dart-define`, or sign-in fails (see [troubleshooting](#)). To set them programmatically, you can use the following methods.

Passing client IDs in code

You can pass the client IDs directly when initializing the Google sign-in service:

```
client.auth.initializeGoogleSignIn(  
  clientId: '<platform_client_id>.apps.googleusercontent.com',  
  serverClientId: '<web_client_id>.apps.googleusercontent.com',  
);
```

This approach is useful when you need different client IDs per platform and want to manage them in your Dart code.

Using environment variables

Alternatively, you can pass client IDs during build time using the `--dart-define` option. The Google sign-in provider supports the following environment variables:

- `GOOGLE_CLIENT_ID`: The platform-specific OAuth client ID
- `GOOGLE_SERVER_CLIENT_ID`: The server (web application) OAuth client ID

If `clientId` and `serverClientId` values are not supplied when initializing the service, the provider will automatically fetch them from these environment variables.

Example usage:

```
flutter run \  
-d "<device>" \  
--dart-define="GOOGLE_CLIENT_ID=<platform_client_id>.apps.googleusercontent.com" \  
--dart-define="GOOGLE_SERVER_CLIENT_ID=<web_client_id>.apps.googleusercontent.com"
```

This approach is useful when you need to:

- Manage separate client IDs for different platforms (Android, iOS, web) in a centralized way
- Avoid committing client IDs to version control
- Configure different credentials for different build environments (development, staging, production)

TIP

You can also set these environment variables in your IDE's run configuration or CI/CD pipeline to avoid passing them manually each time.

Configuring the web redirect URI

You can pass the web redirect URI to `initializeGoogleSignIn` via `--dart-define`. This is useful when building for different environments (development, staging, production) without changing `client.dart`:

```
if (kIsWeb) {  
  client.auth.initializeGoogleSignIn(  
    clientId: const String.fromEnvironment('GOOGLE_CLIENT_ID'),  
    redirectUri: const String.fromEnvironment('GOOGLE_WEB_REDIRECT_URI'),  
  );  
} else {
```

```
client.auth.initializeGoogleSignIn();  
}
```

```
flutter run -d chrome \  
  --dart-define="GOOGLE_CLIENT_ID=<web_client_id>.apps.googleusercontent.com" \  
  --dart-define="GOOGLE_WEB_REDIRECT_URI=<your_redirect_uri>"
```

Use the redirect URI that matches the environment you are building for: the integrated-route URL (e.g., `http://localhost:8082/auth/callback`) when Serverpod serves your web app, or your production URL (e.g., `https://my-awesome-project.serverpod.space/auth/callback`). For `flutter run -d chrome`, where the app runs on its own origin, use the [separately-hosted flow](#) instead.

Separately-hosted Flutter web

Use this flow when your Flutter web app and Serverpod are on different origins. Common cases: `flutter run -d chrome` locally with Serverpod on a separate port, or a CDN-hosted Flutter build with a separate API server.

1. Place a static `auth.html` file in your Flutter project's `web/` folder. A single copy is shared across every identity provider that uses an OAuth2 redirect, so create it once. Follow [Web callback page](#) (`auth.html`) in the authentication setup guide.
2. Run Flutter on a fixed port. The examples use `49660`, but any free port works, as long as you keep it consistent everywhere:

```
flutter run -d chrome --web-port=49660
```

3. Update the [server OAuth client](#) with **Flutter's dev server origin** (not Serverpod's, since Flutter serves `auth.html`):
 - **Authorized JavaScript origins:** `http://localhost:49660` locally, `https://app.example.com` in production.
 - **Authorized redirect URIs:** `http://localhost:49660/auth.html` locally, `https://app.example.com/auth.html` in production.
4. Pass the same URL to `initializeGoogleSignIn` via the `redirectUri` argument instead of the route URL.

Customize the sign-in button

See [Styling the buttons](#) for how the `GoogleSignInWidget` parameters interact with the `buttonStyle` set on `SignInWidget`.

INFO

The `SignInWidget` uses the `GoogleSignInWidget` internally to display the Google sign-in flow. You can also supply a custom `GoogleSignInWidget` to the `SignInWidget` to override the default behavior.

```
SignInWidget(  
  client: client,  
  googleSignInWidget: GoogleSignInWidget(  
    client: client,  
    // Shape and label survive inside SignInWidget, unless its buttonStyle  
    // sets them. Brand colors do not.  
    shape: SignInButtonShape.rounded,  
    text: SignInButtonTextVariant.signInWith,  
    // A custom widget replaces the built-in handling, so pass your own callbacks.  
    onAuthenticated: () { /* ... */ },  
    onError: (error) { /* ... */ },  
  ),  
)
```

Using the `GoogleSignInWidget`

The `GoogleSignInWidget` handles the complete Google sign-in flow for iOS, Android, and web.

You can customize the widget's appearance and behavior:

```
GoogleSignInWidget(  
  client: client,  
  // Button customization. The values shown are the defaults.  
  style: GoogleButtonStyle.outline, // or filledBlue, filledBlack  
  size: SignInButtonSize.large, // or medium, small  
  text: SignInButtonTextVariant.continueWith, // or signInWith, signUpWith, signIn  
  shape: SignInButtonShape.pill, // or rounded, rectangular  
  logoAlignment: SignInButtonLogoAlignment.center, // or left  
  minimumWidth: 240, // at most 400  
  textStyle: null, // TextStyle for the label  
  
  // Scopes to request from Google  
  // These are the default scopes, you can add additional scopes as needed.  
  scopes: const [  
    'https://www.googleapis.com/auth/userinfo.email',  
    'https://www.googleapis.com/auth/userinfo.profile',  
  ],  
  
  // Whether to attempt lightweight sign-in (Android and iOS only)  
  attemptLightweightSignIn: false,  
  
  onAuthenticated: () {  
    // Do something when the user is authenticated.  
    //  
    // NOTE: You should not navigate to the home screen here, otherwise
```

```

    // the user will have to sign in again every time they open the app.
  },
  onError: (error) {
    // Handle errors
  },
)

```

Build a custom UI with GoogleAuthController

For more control over the UI, you can use the `GoogleAuthController` class, which provides all the authentication logic without any UI components. This allows you to build a completely custom authentication interface.

```

import 'package:serverpod_auth_idp_flutter/serverpod_auth_idp_flutter.dart';

final controller = GoogleAuthController(
  client: client,
  onAuthenticated: () {
    // Do something when the user is authenticated.
    //
    // NOTE: You should not navigate to the home screen here, otherwise
    // the user will have to sign in again every time they open the app.
  },
  onError: (error) {
    // Handle errors
  },
  attemptLightweightSignIn: false,
  scopes: const [
    'https://www.googleapis.com/auth/userinfo.email',
    'https://www.googleapis.com/auth/userinfo.profile',
  ],
);

// Initiate sign-in
await controller.signIn();

```

NOTE

On web, sign-in always runs through the OAuth2 redirect flow. Call `initializeGoogleSignIn` with both `clientId` and `redirectUri` before calling `signIn()`. When either value cannot be resolved, `initializeGoogleSignIn` throws an `ArgumentError`. Skipping the call entirely leaves the controller in the error state after `signIn()`. Set them up as described in [Web setup](#).

GoogleAuthController state management

Your widget should render the appropriate UI based on the `state` property of the controller. You can also use the below state properties to build your UI:

```
// Check current state
final state = controller.state; // GoogleAuthState enum

// Check if loading
final isLoading = controller.isLoading;

// Check if authenticated
final isAuthenticated = controller.isAuthenticated;

// Get error message
final errorMessage = controller.errorMessage;

// Listen to state changes
controller.addListener(() {
  setState(() {
    // Rebuild UI when controller state changes
  });
});
```

GoogleAuthController states

- `GoogleAuthState.initializing` - Controller is initializing.
- `GoogleAuthState.idle` - Ready for user interaction.
- `GoogleAuthState.loading` - Processing a sign-in request.
- `GoogleAuthState.error` - An error occurred.
- `GoogleAuthState.authenticated` - Authentication was successful.

Related

- [Setup](#): configure the Google Auth Platform and register the identity provider.
- [Troubleshooting](#): fix common Google sign-in errors.
- [UI components](#): style the sign-in buttons and localize the built-in UI.
- [Working with users](#): react to user creation and manage user data.